

BÀI THỰC HÀNH: CẤU TRÚC DỮ LIỆU ĐỒ THỊ (GRAPH)

1. MỤC TIÊU BÀI THỰC HÀNH

- Hiểu khái niệm và cấu trúc của đồ thị (graph), phân biệt vô hướng – có hướng, có trọng số – không trọng số.
- Biết các cách biểu diễn đồ thị trong C++: danh sách kề, ma trận kề.
- Cài đặt và thực hiện các thuật toán duyệt đồ thị: DFS và BFS.
- Ứng dụng đồ thị để giải các bài toán thực tế như tìm đường đi, mạng xã hội, sơ đồ luồng công việc.

2. CÁC VÍ DỤ MINH HỌA (CHƯƠNG TRÌNH HOÀN CHỈNH)

Ví dụ 1: Biểu diễn đồ thị bằng danh sách kề

```
#include <iostream>
#include <vector>
using namespace std;

class Graph {
    int V;
    vector<vector<int>>> adj;

public:
    Graph(int v) {
        V = v;
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void printGraph() {
        for (int i = 0; i < V; ++i) {
            cout << "Đỉnh " << i << ": ";
            for (int v : adj[i])
                cout << v << " ";
        }
    }
};
```

```

        cout << endl;
    }
}
};

int main() {
    Graph g(5);
    g.addEdge(0, 1);
    g.addEdge(0, 4);
    g.addEdge(1, 2);
    g.addEdge(1, 3);
    g.addEdge(1, 4);
    g.addEdge(2, 3);
    g.addEdge(3, 4);
    g.printGraph();
    return 0;
}

```

Ví dụ 2: Duyệt đồ thị bằng DFS (Đệ quy)

```

#include <iostream>
#include <vector>
using namespace std;

class Graph {
    int V;
    vector<vector<int>>> adj;

    void DFSUtil(int v, vector<bool>& visited) {
        visited[v] = true;
        cout << v << " ";
        for (int u : adj[v]) {
            if (!visited[u])
                DFSUtil(u, visited);
        }
    }

public:
    Graph(int v) {
        V = v;
        adj.resize(V);
    }
}

```

```

void addEdge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

void DFS(int start) {
    vector<bool> visited(V, false);
    cout << "Duyệt DFS từ đỉnh " << start << ": ";
    DFSUtil(start, visited);
    cout << endl;
}
};

int main() {
    Graph g(5);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 3);
    g.addEdge(2, 4);
    g.DFS(0);
    return 0;
}

```

Ví dụ 3: Duyệt đồ thị bằng BFS (Hàng đợi)

```

#include <iostream>
#include <vector>
#include <queue>
using namespace std;

class Graph {
    int V;
    vector<vector<int>> adj;

public:
    Graph(int v) {
        V = v;
        adj.resize(V);
    }

    void addEdge(int u, int v) {

```

```

        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void BFS(int start) {
        vector<bool> visited(V, false);
        queue<int> q;
        visited[start] = true;
        q.push(start);

        cout << "Duyệt BFS từ đỉnh " << start << ": ";
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            cout << v << " ";
            for (int u : adj[v]) {
                if (!visited[u]) {
                    visited[u] = true;
                    q.push(u);
                }
            }
        }
        cout << endl;
    }
};

int main() {
    Graph g(5);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 3);
    g.addEdge(2, 4);
    g.BFS(0);
    return 0;
}

```

3. NỘI DUNG BÀI THỰC HÀNH

Bài 1: Biểu diễn đồ thị bằng danh sách kề

- Nhập số đỉnh, số cạnh từ bàn phím.
- Cho phép nhập từng cạnh và in danh sách kề ra màn hình.

Bài 2: Duyệt đồ thị bằng DFS

- Cài đặt hàm DFS sử dụng đệ quy.
- In ra thứ tự các đỉnh được duyệt.
- Thử nghiệm với nhiều đồ thị khác nhau.

Bài 3: Duyệt đồ thị bằng BFS

- Cài đặt hàm BFS dùng hàng đợi.
- Duyệt từ một đỉnh do người dùng chọn.
- So sánh thứ tự duyệt giữa DFS và BFS.

Bài 4 (nâng cao): Đếm số thành phần liên thông

- Với mỗi đỉnh chưa duyệt, gọi DFS/BFS.
- Đếm số lần gọi → chính là số thành phần liên thông trong đồ thị.

Bài 5 (ứng dụng): Mô phỏng mạng xã hội

- Mỗi đỉnh là một người dùng.
- Một cạnh biểu thị mối quan hệ bạn bè.
- Cho biết có bao nhiêu nhóm bạn bè (liên thông).
- Tìm tất cả bạn bè trong cùng nhóm với người dùng X.